

GRAPHPTC: MODELING PROGRAMMATIC TOOL CALLING AS A DYNAMIC EXECUTION GRAPH

Anonymous authors

Paper under double-blind review

ABSTRACT

Tool calling has become the primary way large language models act on external environments, and its form is shifting from issuing one tool call per model turn toward Programmatic Tool Calling (PTC), where a model writes a program that composes many tool calls, control flow, and intermediate data processing inside a single block executed in a persistent runtime. PTC reduces model-tool interaction rounds and expresses in-block control flow that a per-call protocol cannot, yet across program blocks its execution history remains a linear sequence of code, standard output, and errors. This linear history leaves cross-block dependencies unrepresented, execution progress implicit, and decision feedback unstructured, so the agent repeatedly reconstructs prior results and their relations by re-reading raw text. In this paper we introduce GRAPHPTC, which preserves the programmatic expressiveness of PTC and maintains a task-level dynamic execution graph from real running traces. GRAPHPTC models the internal logic of each block as a typed subgraph of tool effects, artifacts, states, and failures, and models cross-block dependencies as a multi-relation graph over which the agent declares intent and chooses to continue, patch a failed step, or replan a dependency path. Experiments on seven tool-use benchmark settings with four backbone models show that GRAPHPTC improves over both Direct Tool Calling and PTC on every benchmark, reaches a given accuracy with 1.5 to 3 times fewer interaction rounds than Direct Tool Calling, and widens its advantage on complex long-horizon tasks.

1 INTRODUCTION

Tool calling is the primary interface through which large language models (LLMs) act on external environments, and its form has been evolving. Early agents issue a single tool call per model turn and read its result before deciding the next call, following the reasoning-and-acting paradigm (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024). Recent systems instead adopt *Programmatic Tool Calling* (PTC), where the model writes a program that composes many tool calls with loops, conditionals, and intermediate data processing, and the program runs in a persistent runtime whose variables carry across turns (Gao et al., 2023; Wang et al., 2024; Hugging Face, 2025; Anthropic, 2025). By packing a coherent phase of work into one executable block, PTC reduces the number of model-tool interaction rounds, expresses control flow that a call-per-turn protocol cannot, and lets a later block reuse variables produced by an earlier one. However, the expressiveness that PTC gains inside a block is not matched by how it organizes work across blocks.

Across blocks, the history that a PTC agent conditions on is a flat sequence of code, standard output, and error text ordered by time, which is a lossy record of what execution did. A single block on the AppWorld benchmark (Trivedi et al., 2024) issues five to ten tool calls whose read or write effects and produced values are flattened into one output string, and a task spans more than one hundred such values across a dozen blocks. When the agent later needs to know whether a write took effect, which earlier value a new step depends on, or whether two differently phrased queries returned the same content, it has to reconstruct these relations by re-reading raw text. This difficulty stems from a representation that stores execution as concatenated strings and never as the dependency structure that execution induces.

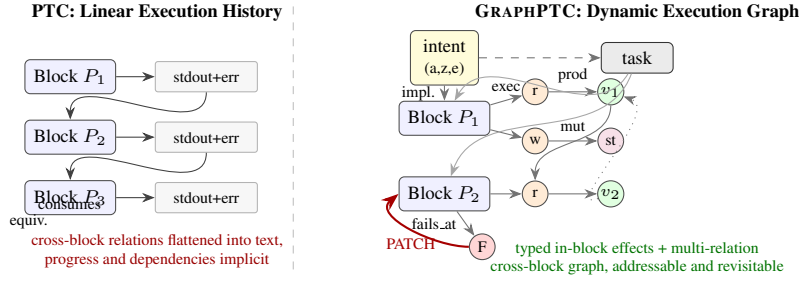


Figure 1: PTC records execution as a linear stream of code and output where cross-block dependencies and progress are lost, whereas GRAPHPTC maintains a dynamic execution graph that models the internal effects of each block (tool calls with read r or write w effects, artifacts v , states st , failures F) and cross-block dependencies (consumes, equivalence, supersession), over which a declared intent selects continue, patch, replan, or answer.

The concatenated representation produces three limitations that build on one another. At the representation layer, the history is a *linear execution context*, where call provenance, artifact consumption, and state versioning have no unified form, so the agent rebuilds prior results and their relations from scratch. At the analysis layer, feedback carries *implicit execution progress*, because it never relates what a block intended to achieve to what it actually changed, and a block that raises no error can still re-fetch an existing result without advancing the task. At the decision layer, standard output and errors give *unstructured decision feedback* that describes only the current block, leaving the agent to infer from scattered history whether to proceed, fix the program, or change strategy, and it easily repeats a prior attempt or extends an unproductive path.

To address these limitations, we introduce GRAPHPTC, which keeps the programmatic expressiveness of PTC and maintains a task-level *dynamic execution graph* built incrementally from real running traces. As shown in Figure 1, GRAPHPTC changes two structural aspects of PTC. First, it models the internal logic of each block, so a code execution becomes a typed subgraph of tool calls with read or write effects, produced artifacts, state versions, and localized failures instead of an opaque output string. Second, it models cross-block dependency as a multi-relation graph instead of a linear text stream, linking equivalent results, reused artifacts, and superseded states so that history becomes addressable. Building on these two changes, GRAPHPTC runs three stages. Graph-based execution modeling projects each trace into a shared typed graph and links cross-block dependencies. Graph-based progress assessment binds a declared intent to the graph change it produces and separates new results from repeated ones. Graph-based action selection turns the graph into a bounded summary over which the agent chooses to continue, patch a failed step, replan a dependency path, or answer.

We evaluate GRAPHPTC on seven tool-use benchmark settings with four backbone LLMs. GRAPHPTC improves over both Direct Tool Calling and PTC on every benchmark, by 7.4 points over PTC on average, and it reaches a given accuracy with 1.5 to 3 times fewer interaction rounds than Direct Tool Calling. Its advantage also widens with task complexity, reaching 43 points over Direct Tool Calling on the hardest tasks, and it transfers across closed and open backbones. Our contributions are three-fold.

- **Graph-Structured Tool Calling.** To the best of our knowledge, GRAPHPTC is the first framework that models programmatic tool calling as a task-level dynamic execution graph, capturing in-block tool effects and cross-block dependencies that a linear history discards.
- **Graph-Guided Execution.** We bind a declared intent to the graph change it produces and construct a bounded graph feedback, which lets the agent judge progress from graph effects and treat recovery as an addressable graph operation over continue, patch, replan, and answer.
- **Consistent Empirical Gains.** On seven settings and four backbones, GRAPHPTC surpasses Direct Tool Calling and PTC on the primary metric, widens its advantage on complex long-horizon tasks, and reduces interaction rounds at equal accuracy.

2 RELATED WORK

Tool Calling and Programmatic Agents. Tool-augmented LLMs have progressed from single-call protocols to programmatic execution. Early work teaches models to emit one structured tool call per turn and interleave it with reasoning, as in Toolformer (Schick et al., 2023), ReAct (Yao et al., 2023), ToolLLM (Qin et al., 2024), and Gorilla (Patil et al., 2024), and multi-agent frameworks such as AutoGen (Wu et al., 2023) and MetaGPT (Hong et al., 2024) coordinate several such agents. A parallel line expresses actions as code, starting from program-aided reasoning (Gao et al., 2023; Chen et al., 2023) and maturing into code-as-action agents that emit executable programs calling tools in a live interpreter, including CodeAct (Wang et al., 2024), code-agent libraries (Hugging Face, 2025), and recent programmatic tool calling that runs a model-written program against many tools in one runtime (Anthropic, 2025). These programmatic approaches gain expressiveness and fewer interaction rounds inside a block, yet they still hand the model a linear transcript of code and output across blocks, and the dependency structure that execution induces is left for the model to reconstruct.

Feedback, Reflection, and Memory for Agents. A second line improves how agents use their own history. Reflection methods summarize past outcomes into natural-language advice, as in Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023), but the summary is unstructured text and does not track which concrete result each step produced. Memory systems extend the usable context, from paged memory in MemGPT (Packer et al., 2023) to knowledge-graph and agentic memory stores (Rasmussen et al., 2025; Xu et al., 2025), and cognitive-architecture views organize an agent around explicit state (Sumers et al., 2024). These systems index facts and entities for later retrieval, whereas GraphPTC indexes what an execution did, namely the call provenance, artifact flow, state versioning, and localized failures inside and across code blocks, and turns that structure into the signal that steers the next action.

In this work, we target the execution history of programmatic tool calling, with the goal of turning a linear transcript of code and output into a structure an agent can reason over. Unlike per-call agents that never compose tool calls into programs, and unlike programmatic agents that keep execution history as flat text, GraphPTC adopts a more deliberate design by maintaining a task-level dynamic execution graph that models in-block tool effects and cross-block dependencies, and by driving action selection from a bounded summary of that graph.

3 GRAPHPTC

GraphPTC retains the ability of PTC to organize many tool calls and intermediate data processing into one executable program, and augments it with a task-level dynamic execution graph that is built from real running traces. The design follows three stages that respond one to one to the three limitations of Section 1. Graph-based execution modeling resolves the linear execution context, graph-based progress assessment resolves the implicit execution progress, and graph-based action selection resolves the unstructured decision feedback.

We first fix notation. Let P_t be the program the agent writes at round t , $\mathcal{G}_{t-1}$ the accumulated graph before this round, and τ_t the trace obtained by executing P_t with instrumented tools. When the agent writes P_t it also declares an intent I_t . The runtime projects the trace into the graph, extracts the change, and produces a bounded feedback, written as

$$\mathcal{G}_t = f_{\text{proj}}(\mathcal{G}_{t-1}, I_t, \tau_t), \quad \Delta_t = f_{\text{diff}}(\mathcal{G}_{t-1}, \mathcal{G}_t), \quad F_t = f_{\text{sum}}^B(\mathcal{G}_t, I_t, \Delta_t), \quad (1)$$

where f_{proj} is the trace projection that registers the intent and writes the execution into the graph, f_{diff} extracts the effect of the round, and f_{sum} organizes that effect into decision evidence under a feedback budget B . The agent then produces the next intent and program from the task q , the retained history $\mathcal{H}_t$, and the feedback, as $(I_{t+1}, P_{t+1}) = \pi(q, \mathcal{H}_t, F_t)$, where π denotes the backbone LLM.

3.1 GRAPH-BASED EXECUTION MODELING

This stage turns a time-ordered running record into a typed execution graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ maintained per task, so that result provenance, cross-block consumption, and state evolution obtain one representation. The node set $\mathcal{V}$ covers program blocks, tool calls, artifacts, state versions, and failures,

together with a task node and intent nodes, and the edge set $\mathcal{E}$ carries typed relations for execution, data flow, and state.

Typed Trace Projection. The key departure from PTC is that a code execution is modeled by its internal effects and not by its output string. During execution, tool wrappers record the arguments, results, and effects of every call, and f_{proj} attaches these calls to the current block b_t and supplements block outputs, state accesses, and failures, adding the edges

$$\{b_t \xrightarrow{\text{exec}} c \mid c \in \mathcal{C}_t\} \cup \{c \xrightarrow{\text{prod}} v \mid v \in \mathcal{A}_t\} \cup \{b_t \xrightarrow{\text{fail}} \phi \mid \phi \in \Phi_t\}, \quad (2)$$

where $\mathcal{C}_t$ is the set of calls issued by b_t , $\mathcal{A}_t$ the artifacts they produce, and Φ_t the failures, and every call c carries a typed effect $\varepsilon(c) \in \{\text{read}, \text{write}\}$ with success and state-change flags. Because calls inside loops and conditionals all enter the graph, multi-tool execution within one block becomes analyzable, and by sharing node identifiers a later block links to existing nodes to form one representation across the whole episode.

Cross-Block Dependency Linking. The second departure is that history across blocks is a multi-relation graph and not a linear order. Guided by tool effect contracts, f_{proj} assigns each artifact a normalized identity key $\kappa(\cdot)$ over its content, and links a later artifact to an earlier one when their keys agree,

$$v \xrightarrow{\text{equiv}} v' \quad \text{when} \quad \kappa(v) = \kappa(v'), \quad (3)$$

so two differently phrased queries that return the same content are recognized as equivalent and not as distinct results. Equivalence is conservative and applies only to read-effect calls that the contract marks deterministic, so a write is never treated as a duplicate and a real state change is never masked. Subsequent use of an existing artifact is recorded as consumes when it serves as input and as reuses when execution reuses it directly, and observable state is versioned with supersedes edges between successive writes, so a later call in an application workflow links to an earlier write under the same representation used for artifact dependencies in retrieval. The output is the accumulated graph $\mathcal{G}_t$ holding the current block with its calls, artifacts, states, failures, and cross-block links.

3.2 GRAPH-BASED PROGRESS ASSESSMENT

This stage relates what a block intended to do to what it actually changed, so that new results, repeated results, and target-local stagnation are separated.

Intent Registration and Binding. Here the purpose of a block becomes a first-class object bound to its observed effect. When the agent submits a program it declares an intent

$$I_t = (a_t, z_t, e_t), \quad (4)$$

where a_t is the action type, z_t the target node, and e_t the expected change, and the target may be the task node or an existing subgoal. Before execution f_{proj} registers an intent node with a targets edge to z_t and snapshots the current artifacts and states, and after execution it binds the intent to the executed block through an implemented_by edge and attaches an action receipt, so later analysis can locate which attempt produced which result along the same target.

Graph Delta and Stagnation Analysis. Progress is defined on graph effects instead of on new output text. From the executed block, f_{diff} collects the produced artifacts $\mathcal{A}_t$, assigns to $\mathcal{E}_t$ those carrying an equiv in-edge, and defines the new artifacts and the progress indicator as

$$\mathcal{N}_t = \mathcal{A}_t \setminus \mathcal{E}_t, \quad p_t = \mathbb{1}[\mathcal{N}_t \neq \emptyset \vee \mathcal{S}_t \neq \emptyset], \quad (5)$$

where $\mathcal{S}_t$ is the set of state effects linked by writes or mutates, so allocating a new result node or reusing a cached one does not count as new progress. A target-local stagnation count then follows the recurrence

$$s_t(z_t) = \begin{cases} 0, & p_t = 1, \\ s_{t-1}(z_t) + 1, & p_t = 0, \end{cases} \quad (6)$$

where counts on other targets stay independent, so the agent is told when a target stops advancing even though no error is raised. The count is an advisory input the agent may override, and a single non-advancing block changes nothing, since a plan change is only suggested once the count passes a threshold. The output is the round effect Δ_t , the receipt bound to the declared action, and the stagnation count.

Algorithm 1 GRAPHPTC performs graph-guided programmatic tool calling.

Require: task q , tools, feedback budget B

```

1: initialize graph  $\mathcal{G} \leftarrow \{\text{task node}\}$ , feedback  $F \leftarrow \emptyset$ 
2: while budget remains do
3:    $(I, P) \leftarrow \pi(q, \mathcal{H}, F)$  {declare intent  $I = (a, z, e)$  and program}
4:   if  $a$  is ANSWER then
5:     return the answer
6:   end if
7:   register  $I$  and snapshot  $\mathcal{G}$ 
8:    $\tau \leftarrow \text{execute } P \text{ with instrumented tools}$ 
9:    $\mathcal{G} \leftarrow f_{\text{proj}}(\mathcal{G}, I, \tau)$  {typed projection and cross-block linking, Sec. 3.1}
10:   $\Delta \leftarrow f_{\text{diff}}(\mathcal{G})$  over new artifacts  $\mathcal{N}$ , state effects  $\mathcal{S}$ , failures, stagnation {Sec. 3.2}
11:   $F \leftarrow f_{\text{sum}}^B(\mathcal{G}, I, \Delta)$  with opportunity set  $\mathcal{O} = f_{\text{opp}}(\Delta)$  {Sec. 3.3}
12: end while

```

3.3 GRAPH-BASED ACTION SELECTION

This stage turns the graph evidence into a bounded feedback and offers the next action with its reason, and the agent makes the choice.

Bounded Feedback Construction. The agent receives an evidence-focused view of the graph instead of the whole graph. From the accumulated graph, the intent, and the delta, f_{sum} extracts the declared action, its receipt, the new and equivalent artifacts, the state changes, the failure context, and the prior attempts on the target, and reduces them under the budget by pruning list items and then fields,

$$F_t = f_{\text{sum}}^B(\mathcal{G}_t, I_t, \Delta_t), \quad |F_t| \leq B, \quad (7)$$

so the long-horizon record stays at runtime while the model receives only the evidence its current decision needs.

Graph-Guided Action Selection. Recovery is exposed as an addressable graph operation instead of free-form retrying. The runtime turns execution signals into an opportunity set $\mathcal{O}_t = f_{\text{opp}}(\Delta_t)$, where a failure yields a repair opportunity and a stagnation that reaches a threshold yields a replanning opportunity, and the agent selects an action

$$a_{t+1} \in \{\text{CONTINUE}, \text{PATCH}, \text{REPLAN}, \text{ANSWER}\}, \quad (8)$$

where CONTINUE advances a new step, PATCH returns to a failed node and re-executes, REPLAN changes the dependency path, and ANSWER terminates. The first three actions carry a new intent and program and re-enter Eq. 1, so an action label is tied to concrete program behavior. The opportunity rules and the feedback budget are fixed defaults used unchanged across all benchmarks and backbones, so no part of the controller is tuned per task. Algorithm 1 summarizes the loop.

4 EXPERIMENTS

4.1 BENCHMARKS AND SETUP

We evaluate on seven tool-use benchmark settings that span retrieval, question answering, application control, and workflow automation. We choose these benchmarks because they stress the cross-block dependency that GraphPTC targets along complementary axes, they range from short to long horizons, and each carries an executable or fixed-grader evaluation that scores an outcome against a reference. **BrowseComp-Plus** (Wei et al., 2025) measures evidence-seeking browsing over a fixed corpus and reports answer accuracy. **AppWorld** (Trivedi et al., 2024) is an executable world of everyday apps, and we use its normal and challenge splits scored by Task Goal Completion (TGC) and Scenario Goal Completion (SGC). **Agent-Diff** measures stateful multi-step tasks with an assertion-based pass rate. **FanOutQA** (Zhu et al., 2024) measures multi-hop, multi-document question answering with a judge accuracy. **FRAMES** (Krishna et al., 2025) measures fact retrieval and reasoning with a judge accuracy. **AutomationBench** measures business workflow automation with a task completion rate. The two AppWorld splits give seven evaluation settings over six benchmarks, and with four backbones this forms 28 model-setting pairs.

Method	BrowseComp-Plus (Acc)	AppWorld-Normal (TGC)	AppWorld-Challenge (TGC)	Agent-Diff (Pass)	FanOutQA (Judge)	FRAMES (Judge)	Automation-Bench (Pass)
<i>GPT-5.6-Sol</i>							
Direct Tool Calling	73.49	82.10	81.80	51.34	55.81	77.31	20.83
PTC	80.00	89.90	90.60	39.73	52.58	73.42	29.17
GraphPTC	80.60	91.70	93.00	51.49	60.00	79.00	31.00
<i>Claude-Opus-5</i>							
Direct Tool Calling	43.98	94.00	80.80	75.14	77.74	84.95	28.67
PTC	10.96	59.50	79.10	68.06	76.77	85.19	32.50
GraphPTC	49.04	95.80	95.00	79.17	78.71	85.56	33.50
<i>DeepSeek-V4-Pro</i>							
Direct Tool Calling	4.58	81.00	65.70	70.09	57.42	81.43	21.50
PTC	3.49	88.70	80.10	72.47	58.06	80.83	17.00
GraphPTC	13.37	89.90	95.20	73.96	62.26	81.55	22.83
<i>Kimi-K3</i>							
Direct Tool Calling	8.92	72.00	46.50	77.53	49.03	73.54	27.50
PTC	2.89	85.70	83.20	78.12	28.71	75.85	28.00
GraphPTC	10.48	86.30	84.90	78.72	50.65	76.46	28.83

Table 1: Main results on seven tool-use benchmark settings across four backbones. Each column reports the primary metric of one setting, with answer accuracy for BrowseComp-Plus, Task Goal Completion (TGC) for the two AppWorld splits, pass rate for Agent-Diff and AutomationBench, and judge accuracy for FanOutQA and FRAMES. **Bold** marks the best method per backbone and benchmark. GraphPTC improves over both Direct Tool Calling and PTC on every benchmark.

We instantiate three methods under one shared protocol. *Direct Tool Calling* uses native per-call function calling, *PTC* uses programmatic tool calling with a persistent runtime and no graph, and *GraphPTC* adds the dynamic execution graph of Section 3. All three share the same tools, prompts, and budgets on each benchmark, and the feedback budget of GraphPTC is held fixed across all benchmarks and backbones, so a gain reflects the graph and not per-setting tuning. The four backbones, GPT-5.6-Sol, Claude-Opus-5, DeepSeek-V4-Pro, and Kimi-K3, cover closed and open, dense and mixture-of-experts models.

4.2 MAIN RESULTS

Capability on Interactive Application Control. We evaluate GRAPHPTC on the AppWorld normal and challenge splits to assess its *long-horizon application control*, where a task drives many app calls whose write effects must persist and compose into a final goal. As shown in Table 1, GRAPHPTC improves over both Direct Tool Calling and PTC on both splits for every backbone, and its margin over PTC grows from the normal split to the challenge split. We attribute this pattern to modeling state as typed nodes, so a later block links to an earlier write instead of re-deriving it from text, and Section 5 tests this attribution directly. Overall, GRAPHPTC’s strong results on AppWorld highlight its capability in orchestrating long-horizon application control.

Capability on Multi-Hop Retrieval and Reasoning. We evaluate GRAPHPTC on BrowseComp-Plus, FanOutQA, and FRAMES to assess its *multi-hop retrieval and reasoning*, where an answer depends on composing evidence gathered across many calls. As shown in Table 1, GRAPHPTC improves over both baselines on all three benchmarks for every backbone, and it recovers the collapsed PTC accuracy of Claude-Opus-5 on BrowseComp-Plus to a level above Direct Tool Calling. This recovery reflects cross-block dependency linking, which marks equivalent results and lets the agent stop re-issuing queries whose content it already holds. Overall, GRAPHPTC’s strong results on these benchmarks highlight its capability in composing multi-hop evidence under programmatic tool calling.

Capability on Stateful Workflow Automation. We evaluate GRAPHPTC on Agent-Diff and AutomationBench to assess its *stateful workflow automation*, where a task completes only when a chain of dependent write operations all take effect. As shown in Table 1, GRAPHPTC improves over both baselines on both benchmarks for every backbone, and it turns the case where PTC underperforms even Direct Tool Calling on DeepSeek-V4-Pro back into a gain. Here progress assessment separates a genuine state change from a re-executed no-op, so the agent advances the workflow instead of re-

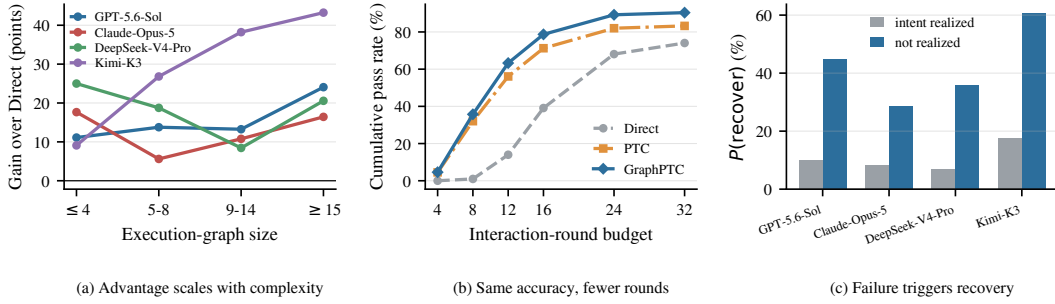


Figure 2: Analyses of GraphPTC on AppWorld. (a) The gain over Direct Tool Calling is positive in every complexity bin and widens toward the most complex tasks. (b) Along the accuracy against interaction-round frontier, GraphPTC reaches the accuracy of the baselines with fewer rounds. (c) An intent that is not realized sharply raises the probability that the agent issues a recovery action, so the progress signal is used.

Configuration	GPT-5.6-Sol	Claude-Opus-5	DeepSeek-V4-Pro	Kimi-K3	Avg
GraphPTC (full)	69.5	73.8	62.7	59.5	66.4
– cross-block graph	65.1	58.9	57.2	54.6	58.9
– cross-block graph and programs	63.2	69.3	54.5	50.7	59.5

Table 2: Ablation of the two structural changes of GraphPTC, reported as the seven-setting average per backbone. **Bold** marks the full model. Removing the cross-block graph and removing the programmatic runtime each cause a comparable drop, indicating the two changes are complementary.

stating a completed step. Overall, GRAPHPTC’s strong results on Agent-Diff and AutomationBench highlight its capability in stateful workflow automation.

Across the panel, GRAPHPTC improves over both Direct Tool Calling and PTC on every benchmark, by 7.4 points over PTC and 6.9 points over Direct Tool Calling on average, with consistent gains on four heterogeneous backbones.

5 ANALYSES

We analyze GraphPTC along its two structural changes, the modeling of in-block logic and the modeling of cross-block dependency as a graph, and study how each converts into tool-calling behavior. The ablation, the efficiency study, and the capability grouping span all seven benchmarks, whereas the studies that read the internal execution graph, namely recovery, reliability, and complexity scaling, use the AppWorld splits because AppWorld exposes the full per-episode graph.

5.1 ABLATION STUDY ON GRAPH MODELING

To separate the contribution of the two structural changes, we ablate them and report the seven-setting average of each backbone. Removing the cross-block graph and its graph-guided recovery reduces GraphPTC to PTC, and further removing the programmatic runtime reduces it to Direct Tool Calling. As shown in Table 2, removing the graph layer costs 7.4 points on average and removing the programs costs a comparable amount, so the in-block and cross-block modeling contribute complementary value. To isolate the in-block modeling on its own, on a representative setting we replace the typed graph feedback with a plain success flag and raw output while holding every other component fixed, and the completion rate drops by several points. Together with the four-backbone ablation above, this indicates that the typed execution graph, and not the programmatic runtime alone, is what lets the agent judge whether a write took effect or where a failure occurred.

5.2 ADVANTAGE OF NON-LINEAR RECOVERY

To test whether modeling cross-block dependency as a graph enables recovery a linear history cannot, we study how the agent uses the actions that return to a specific node. We measure the probability that the agent issues a recovery action conditioned on whether the previous block realized its

declared intent, and on the tasks where GraphPTC uses recovery we compare its pass rate with PTC on the same task identifiers. As shown in Figure 2(c), an unrealized intent raises the recovery probability by three to five times across backbones, so the declared-and-verified intent is an actionable signal that the agent responds to. On the same task identifiers GraphPTC exceeds PTC, and because this comparison fixes the task and differs mainly in whether the agent can return to a failed node or must append the next block, we read the gap as largely driven by non-linear recovery. Recovery is also local and immediate, since across backbones a failed block is followed by a successful one within a single block in 90% of cases, so the agent repairs the failed step instead of re-running the whole block. This shows that treating recovery as an addressable graph operation, instead of free-form retrying, is where the cross-block graph pays off.

5.3 ADVANTAGE ON COMPLEX LONG-HORIZON TASKS

To locate where the advantage of GraphPTC arises, we bin every AppWorld task by the number of program blocks in its execution graph and measure the gain of GraphPTC over Direct Tool Calling within each bin. As shown in Figure 2(a), the gain is positive in every bin for every backbone, and it widens toward the hardest tasks, rising monotonically on GPT-5.6-Sol and Kimi-K3 and staying positive without a monotone trend on the other two. This trend follows from the representation, since a larger task graph holds more intermediate results and more failure-prone steps, so both the addressable artifacts of in-block modeling and the local recovery of cross-block modeling matter more as a linear history grows harder to reconstruct. The gain therefore never trails a linear history and is largest on the long-horizon tasks that motivate programmatic tool calling.

5.4 EFFICIENCY AND TOKEN ECONOMY

To check whether the graph improves efficiency and not only accuracy, we measure the interaction rounds each method spends and trace the accuracy it reaches under a round budget. On AutomationBench, GraphPTC spends 54 to 64% fewer model rounds than Direct Tool Calling across the four backbones while raising the completion rate, because one block carries several dependent calls whose intermediate values are registered as artifacts and reused (per-backbone rounds in Appendix C). As shown in Figure 2(b), on AppWorld the same effect appears along the accuracy against round frontier, where GraphPTC reaches the accuracy of Direct Tool Calling with 1.5 to 3 times fewer rounds and stays on or above the PTC frontier at every budget. The result confirms that graph-guided execution reduces test-time interaction cost, which follows from batching dependent calls within a block and reusing addressable artifacts across blocks instead of rebuilding them from a transcript. The graph is maintained by the runtime and enters the model only through the bounded feedback, so the model input per round stays independent of graph size and the round savings are not offset by a growing context. As shown in Figure 3, for every backbone GraphPTC reaches the highest AppWorld-Challenge accuracy while reading 1.3 to 2.3 times fewer input tokens than Direct Tool Calling, whose per-call transcript re-sends the growing history on each turn, and the same ordering holds on all six benchmarks with logged usage (Appendix C).

5.5 ROBUSTNESS ACROSS BACKBONE CAPABILITY

To examine how the benefit depends on backbone capability, we group the four backbones by their Direct Tool Calling average into stronger backbones, GPT-5.6-Sol and Claude-Opus-5, and weaker backbones, DeepSeek-V4-Pro and Kimi-K3, and compare the gain that GraphPTC adds over Direct Tool Calling. As shown in Figure 4(a), the weaker backbones gain more than the stronger ones. This follows because the execution graph externalizes the state tracking and dependency bookkeeping that a stronger model partly carries on its own, so a weaker model benefits more when that structure is supplied. At the same time, the graph is what keeps PTC from collapsing on a capable backbone, since Claude-Opus-5 loses accuracy under plain PTC yet reaches the best result under GraphPTC. In effect, externalized execution structure both lifts weaker backbones and stabilizes stronger ones.

5.6 RELIABILITY OF THE INTENT SIGNAL

To assess whether the progress signal the agent acts on is trustworthy, we measure how often a declared expected change is realized in the graph and how often a block failure stays local. As shown

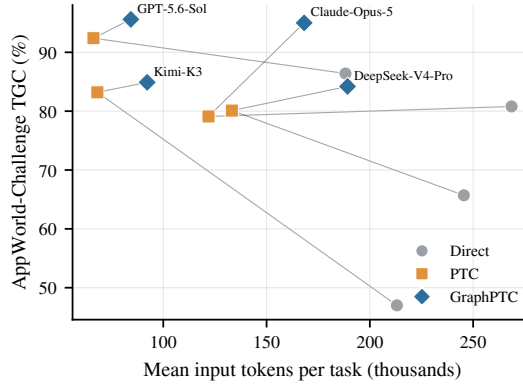


Figure 3: Token economy on AppWorld-Challenge. Each backbone contributes three points joined by a line, and for every backbone GraphPTC reaches the highest Task Goal Completion while reading far fewer input tokens than Direct Tool Calling.

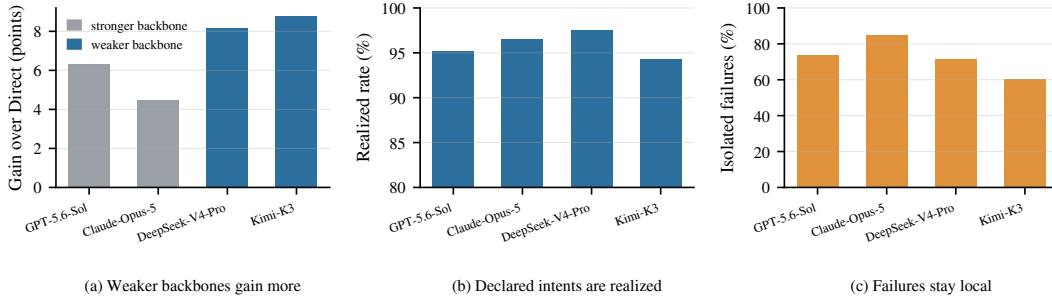


Figure 4: Further analyses of GraphPTC on AppWorld. (a) Grouping backbones by their Direct Tool Calling average, the weaker backbones gain more from the graph than the stronger ones. (b) Across backbones, most declared expected changes are realized in the graph, so the progress signal is grounded in observed effects. (c) Most block failures are isolated, with no further failure within three blocks.

in Figure 4(b), the declared intent is realized in more than nine of ten blocks across backbones, so the signal that drives recovery is grounded in observed effects and not in self-report. As shown in Figure 4(c), most block failures under GraphPTC are isolated, with no further failure in the next three blocks. Under plain PTC a failing task instead repeats the same exception three or more times in up to one fifth of cases, which the graph turns into a targeted return to the failed node. Binding intent to graph effects gives a reliable basis for the continue, patch, and replan decisions, the mechanism that separates GraphPTC from a linear execution history.

5.7 LIMITATIONS

Our study is inference-time and adds no training, so a backbone that cannot follow the intent declaration gains less from the graph. We report one run per setting and treat the agreement of the trend across four heterogeneous backbones and seven settings as the evidence of robustness in place of a per-setting variance estimate. The controlled comparison holds tools, prompts, and budgets fixed, so it isolates the graph but does not combine GraphPTC with reflection or long-term memory, which are orthogonal and could be layered on the same graph.

6 CONCLUSION

We presented GRAPHPTC, a framework that models programmatic tool calling as a task-level dynamic execution graph. By turning the linear execution history of PTC into a typed structure, GRAPHPTC models the internal logic of each program block as tool effects, artifacts, states, and failures, and models cross-block dependencies as a multi-relation graph over which an agent declares intent and chooses to continue, patch a failed step, replan a dependency path, or answer. Experi-

ments on seven tool-use benchmark settings with four backbones show that GRAPHPTC improves over both Direct Tool Calling and PTC on every benchmark, reaches a given accuracy with 1.5 to 3 times fewer interaction rounds than Direct Tool Calling, and widens its advantage on the hardest long-horizon tasks. These results indicate that representing what an execution did, and not only what an agent saw, is what lets a programmatic agent reason over its own history, and future work can extend the execution graph toward cross-task reuse and learned graph-conditioned policies.

REPRODUCIBILITY STATEMENT

The method is specified in Section 3 and Algorithm 1, with node and edge types in Appendix A and further details in Appendix B. All arms share one protocol described in Section 4.1, and the plotting code and the raw numbers behind every analysis figure are released under the `code/` directory.

AI USE STATEMENT

Generative AI tools were used to assist with language editing of the manuscript. All technical content, experiments, and analyses were designed, run, and verified by the authors, who take responsibility for the final content.

REFERENCES

- Anthropic. Code execution with MCP: Building more efficient agents. <https://www.anthropic.com/engineering/code-execution-with-mcp>, 2025.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pp. 10764–10799, 2023.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Hugging Face. smolagents: A barebones library for agents that think in code. <https://github.com/huggingface/smolagents>, 2025.
- Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. In *arXiv preprint arXiv:2310.08560*, 2023.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, 2024.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *International Conference on Machine Learning*, 2024.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Andrew Zhu, Alyssa Hwang, Liam Dugan, and Chris Callison-Burch. Fanoutqa: A multi-hop, multi-document question answering benchmark for large language models. *Annual Meeting of the Association for Computational Linguistics*, 2024.

A NODE AND EDGE TYPES OF THE EXECUTION GRAPH

Table 3 lists the typed nodes and edges maintained in the execution graph. Node types cover the task, program blocks, tool or API calls with read or write effects, artifacts, state versions and state effects, failures, and action intents. Edge types cover execution structure, data flow, state evolution, failure attribution, cross-block equivalence and reuse, and intent binding.

Category	Types
Nodes	TASK, BLOCK, CALL (read/write), ARTIFACT, STATE, FAILURE, INTENT
Execution	contains, executes, produces, fails_at
Data flow	consumes, reuses, derives
State	reads, writes, mutates, supersedes
Cross-block	equivalent_to
Intent	targets, implemented_by

Table 3: Node and edge types of the dynamic execution graph.

B IMPLEMENTATION DETAILS

The execution graph is maintained incrementally with node and edge cursors, so extracting the round delta Δ_t visits only the nodes and edges added in the current round and does not solve a general graph edit distance. Tool wrappers record arguments, results, and effects during execution, and the trace projector attaches these to the current block after execution. Cross-block equivalence uses tool effect contracts to normalize argument and result identities, so cached deterministic reads may be reused while tools with side effects follow their contract. The bounded feedback prunes list items first and then reduces fields under the character budget B , and the full graph is retained at runtime and never sent to the model in whole.

C ADDITIONAL ANALYSES

Scenario Goal Completion on AppWorld. Table 4 reports the secondary AppWorld metric, Scenario Goal Completion, which requires every task in a scenario to succeed together. GraphPTC improves Scenario Goal Completion on both splits for every backbone, and the margin over PTC is wider than on Task Goal Completion, which matches the intuition that a scenario is only as strong as its most dependent step.

Backbone	AppWorld-Normal		AppWorld-Challenge	
	PTC	GraphPTC	PTC	GraphPTC
GPT-5.6-Sol	83.90	85.70	82.00	87.10
Claude-Opus-5	46.40	92.90	69.10	92.10
DeepSeek-V4-Pro	80.40	82.10	66.90	88.50
Kimi-K3	67.90	76.80	66.90	70.50

Table 4: Scenario Goal Completion on AppWorld. **Bold** marks the better method per backbone and split. GraphPTC improves the scenario metric on every backbone and split.

Use of the recovery actions. Table 5 counts how often the agent issues PATCH and REPLAN on AppWorld and the pass rate of the tasks that use them. The recovery actions are used on a large share of tasks, and the tasks that trigger a PATCH still pass in almost all cases, which shows the graph turns a failure into a targeted retry that usually succeeds.

Backbone	PATCH calls	REPLAN calls	Pass rate on PATCH tasks
GPT-5.6-Sol	516	52	100.0
Claude-Opus-5	506	20	97.8
DeepSeek-V4-Pro	586	0	94.7
Kimi-K3	972	3	99.1

Table 5: Use of the recovery actions on AppWorld, summed over the normal and challenge splits. Tasks that trigger a PATCH pass in almost all cases.

Interaction rounds on AutomationBench. Table 6 reports the average model rounds per task on AutomationBench. GraphPTC spends 54 to 64% fewer rounds than Direct Tool Calling on every backbone while reaching a higher completion rate, which complements the AppWorld frontier in the main text and shows the efficiency gain is not specific to one benchmark.

Backbone	Direct Tool Calling	GraphPTC	Fewer rounds
GPT-5.6-Sol	32.4	11.5	64%
Claude-Opus-5	27.0	11.6	57%
DeepSeek-V4-Pro	54.8	20.1	63%
Kimi-K3	30.2	13.9	54%

Table 6: Average model rounds per task on AutomationBench. GraphPTC reaches a higher completion rate with far fewer rounds on every backbone.

Input tokens across benchmarks. Table 7 reports the mean input tokens a task consumes, averaged over the four backbones, for six benchmarks. On every benchmark GraphPTC reads far

fewer input tokens than Direct Tool Calling and only modestly more than PTC, so the token economy shown for AppWorld in the main text holds throughout, since a per-call transcript re-sends the growing history on each turn while the bounded feedback does not.

Benchmark	Direct Tool Calling	PTC	GraphPTC
AppWorld-Normal	151.4k	60.2k	84.1k
AppWorld-Challenge	228.8k	97.3k	133.5k
Agent-Diff	113.2k	34.4k	49.0k
FanOutQA	128.5k	49.5k	63.9k
FRAMES	93.0k	42.2k	55.2k
AutomationBench	324.4k	156.6k	171.9k

Table 7: Mean input tokens per task, averaged over the four backbones. GraphPTC reads far fewer input tokens than Direct Tool Calling on every benchmark while reaching the best accuracy.

Gain by reasoning type on FRAMES. Table 8 breaks the FRAMES gain down by reasoning type for the two closed backbones. The gain is largest on post-processing and temporal reasoning, where an answer must combine results across several calls, which is where cross-block linking removes the most redundant work.

Backbone	Multiple constraints	Numerical	Post-processing	Tabular	Temporal
GPT-5.6-Sol	+1.5	+1.4	+4.7	+1.7	+2.2
Claude-Opus-5	−0.6	+1.4	+0.9	+0.4	+2.2

Table 8: FRAMES gain of GraphPTC over the stronger baseline by reasoning type, in accuracy points. The gain concentrates on post-processing and temporal reasoning.

Where the workflow gain arises. Agent-Diff labels each task by the operations it performs, and Table 9 splits the tasks by whether they update or delete existing state. GraphPTC improves over PTC by 7.2 points on the state-overwriting tasks and by essentially nothing on the tasks with no overwrite, which matches the mechanism, since the state versioning of the graph helps exactly where a later step depends on a mutated state and adds little where there is no state to track.

Task type	#tasks	Direct Tool Calling	PTC	GraphPTC
Update or delete state	1649	63.3	58.3	65.5
No state overwrite	654	79.2	79.0	78.7

Table 9: Agent-Diff pass rate split by whether a task overwrites existing state, aggregated over the four backbones. GraphPTC’s gain concentrates on state-overwriting tasks, where the state versioning of the graph applies.

D CASE STUDIES

Figure 5 draws an example execution graph and marks the three mechanisms discussed below. Each trace is abbreviated to the relevant blocks.

Case 1: cross-block equivalence removes redundant retrieval. On a multi-hop question, block P_1 queries *population of the capital of country X* and block P_3 later queries *capital city X population*. The trace projector assigns both results the same identity key and adds an *equivalent_to* edge (mark (1) in Figure 5), and the feedback reports the second result as equivalent. Under PTC the same content returns as a fresh output string, and the agent issues a third query to reconcile them, whereas under GraphPTC the agent reads the existing artifact and moves to the next hop.

Case 2: a failed write becomes a targeted patch. On an AppWorld task, block P_4 posts a payment whose call raises an error, so the projector adds a *FAILURE* node with a *fails_at* edge and the feedback marks the intent unrealized (mark (2)). The agent declares *PATCH* on the same target, fixes the malformed field, and re-executes only that step. Under PTC the failure is one line inside a long output, and the agent re-runs the whole block, which re-issues the calls that had already succeeded.

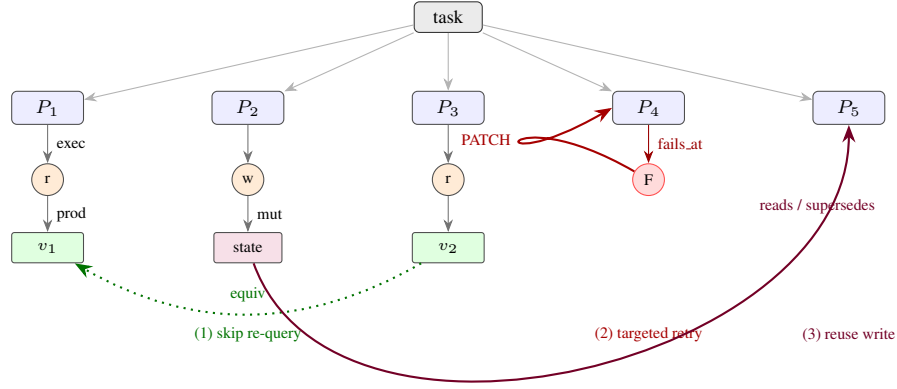


Figure 5: An example execution graph built by GraphPTC. Blocks P_i are projected into typed calls (read r , write w), artifacts v , states, and failures F . (1) block P_3 produces v_2 that an `equivalent_to` edge links to the earlier v_1 , so the agent skips a redundant query. (2) a failed call in P_4 adds a `FAILURE` the agent repairs with a targeted `PATCH` instead of re-running the block. (3) block P_5 reads the state written by P_2 through the `supersedes` chain instead of re-deriving it.

Case 3: a later block links to an earlier write instead of re-deriving it. On a workflow task, block P_2 updates a contact record and block P_5 needs the updated field. The state is versioned with a `supersedes` edge, so the feedback points block P_5 at the current version and the agent reads it directly (mark (3)). Under PTC the agent re-fetches the record to be sure of its value, which adds a round and risks reading a stale copy from earlier in the transcript.